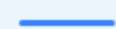


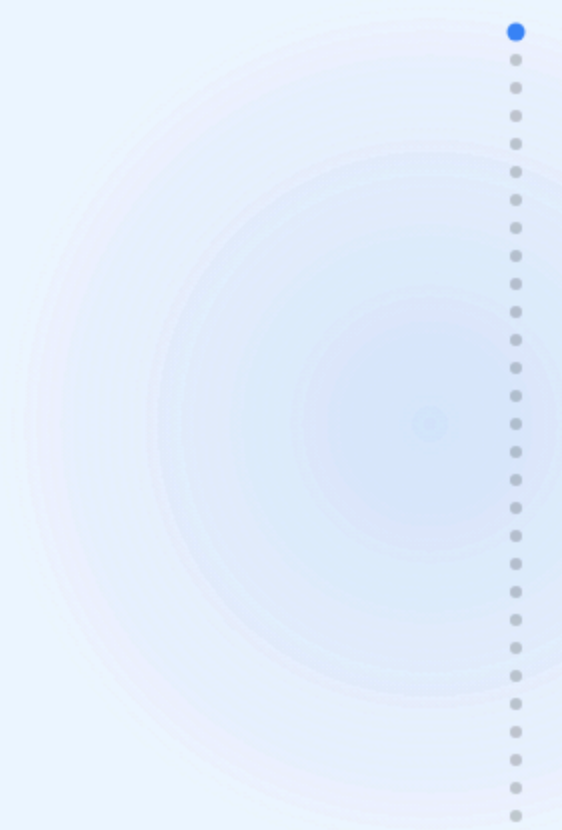
ATO 0 • REVISÃO

Antes de começar:

você ainda lembra?



Engenharia de Software • TEI 162 • 10/abr/2026



Turma responde — sem olhar o caderno.

01 O que diferencia um **requisito funcional** de um não-funcional?

02 Qual a diferença entre **User Story** e **Caso de Uso**?

03 Quando um requisito pode ser considerado "**pronto**"?

CONSOLIDAÇÃO — FOTOGRAFA ESTE SLIDE

Tudo que você viu no N1, numa tela só.



Req. Funcional
O que o sistema FAZ



Req. Não-Funcional
Desempenho, segurança, usabilidade



User Story
Como [usuário], quero [ação]...

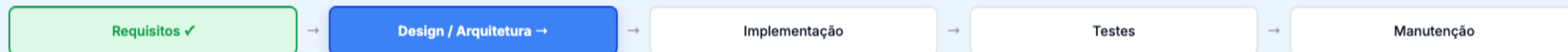


Caso de Uso
Ator + sistema + fluxo completo



Validação
Critérios de aceite + negociação

Onde estamos no ciclo de vida?



No Cascata, depois dos requisitos vem o **Design**. Scrum ou Cascata — em algum momento você precisa *desenhar* o sistema antes de codificá-lo.

ATO 1 · O CASE



Nubank

Banco digital · Fundado 2013 · +100 milhões de clientes

Como nasce um produto de R\$ 300 bilhões?

Quem tem conta no Nubank? 🖐️ · Quem já abriu conta em banco físico?

BRASIL • 2013

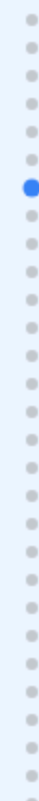
"Abrir uma conta bancária levava
40 dias, 12 documentos e **3**
visitas à agência."

David Vélez passou por isso. Em vez de reclamar, ele fez o que qualquer engenheiro deveria fazer: **desenhou o problema antes de codificar a solução.**

Como você conta para **50 engenheiros** o que cada parte do sistema faz

— sem usar código?

A resposta que a indústria adotou se chama **UML**.



UNIFIED MODELING LANGUAGE · ISO/IEC 19501 · DESDE 1997

Dois tipos de diagrama UML.

Estruturais — o que o sistema É

- **Diagrama de Classes** ← hoje
- Diagrama de Componentes
- Diagrama de Pacotes
- Diagrama de Objetos

Comportamentais — o que o sistema FAZ

- Diagrama de Sequência ← próxima aula
- Diagrama de Atividades ← próxima aula
- Diagrama de Casos de Uso ← próxima aula
- Diagrama de Estados

Cada classe: uma planta baixa de um objeto.

Conta
- saldo : Float - limite : Float - dataAbertura : Date
+ sacar(valor : Float) + depositar(valor : Float) + consultarSaldo() : Float

- **Nome:** identifica a entidade do mundo real
- **Atributos:** o que o objeto *tem* (estado)
- **Métodos:** o que o objeto *faz* (comportamento)

Quem pode ver o quê?

- **+ Público:** qualquer objeto pode acessar. Ex: `+ sacar()`
- **- Privado:** só a própria classe acessa. Ex: `- saldo : Float`
- **# Protegido:** a classe e suas subclasses. Ex: usado em herança

O `saldo` nunca deveria ser público — qualquer objeto poderia alterá-lo. Por isso é privado, acessado apenas por `consultarSaldo()`.

Associação: como objetos se conhecem.



1 cliente → 1..* Conta



1 conta → 0..* Transação



- **1** — exatamente um

- **1..*** — um ou mais

- **0..1** — zero ou um (opcional)

- ***..*** — muitos para muitos

DEBATE · 1 MINUTO

Conta Corrente e Conta Poupança são coisas **diferentes** — ou a **mesma coisa** com variações?

Defenda sua resposta. Essa decisão separa um diagrama bom de um diagrama ruim.

Duas regras para memorizar.



Herança

"É um tipo de"

ContaCorrente extends Conta

ContaCorrente e ContaPoupança *herdam* atributos e métodos de Conta.



Composição

"É parte de"

Conta ◆— Transação

Se a Conta for deletada, as Transações somem — existência dependente.

DIAGRAMA DE CLASSES · NUBANK — FOTOGRAFA ESTE SLIDE

O esqueleto do Nubank em UML.



Cliente 1 — * Conta · Conta ◄ — * Transação · Conta — 1 Limite · Conta — * ChavePix · Conta — * Cartão

PONTE · ATO 3

Como o engenheiro sabia que precisava de uma classe **Limite** — e não apenas um atributo?

Ele não inventou. Ele descobriu.

A ferramenta que ensina a descobrir o problema certo chama-se **Design Thinking**.



PREVIEW · PRÓXIMA AULA (17/ABR)

Três diagramas comportamentais que você vai usar no projeto.

Sequência

Ordem das mensagens entre objetos ao longo do tempo.

→ fluxo de login, pagamento Pix

Atividade

Fluxos de trabalho com decisões, paralelismo e loops.

→ processo de abertura de conta

Caso de Uso

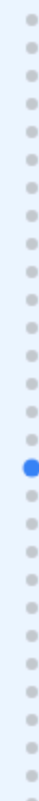
Atores e funcionalidades disponíveis para cada perfil.

→ ator Cliente, ator Banco

A maioria dos times modela a solução **errada** para o problema **errado**.

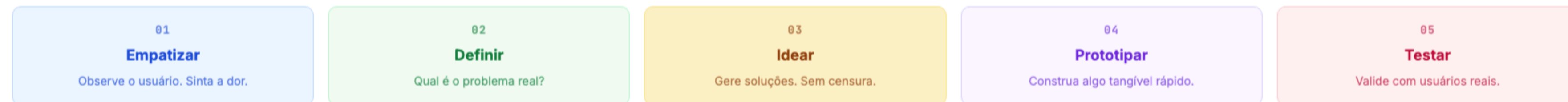
Não porque são ruins — porque nunca foram ensinados a *empatizar* antes de modelar.

Design Thinking é o antídoto.



DESIGN THINKING · PROCESSO

5 fases. Não é linear — é iterativo.



Você volta ao início quando descobre que estava resolvendo o problema errado. Isso é saúde no projeto, não falha.

Como o Nubank usou cada fase.

- **Empatizar:** David foi ao banco e viveu a burocracia na pele. Mapeou a frustração como usuário.
- **Definir:** O problema não é "banco ruim" — é "abrir conta é humilhante e burocrático". Foco no processo.
- **Idear:** 100% digital, sem agência, sem documentos físicos, sem anuidade no cartão.
- **Prototipar + Testar:** → próximo bloco.

O produto surgiu da fase Definir — não da fase Idear. Definir o problema certo foi o diferencial competitivo do Nubank.

Cada fase do DT alimenta uma decisão no Diagrama de Classes.

Empatizar

→ Você descobre os **atores** (quem usa o sistema e como)

Definir

→ Você descobre os **requisitos reais** (não os pedidos literais)

Idear

→ Você propõe as **funcionalidades** → métodos das classes

Prototipar

→ Você cria um **modelo navegável** → rascunho do diagrama

Testar

→ Você **valida o modelo** antes de codificar

PONTE · ATO 4

5 fases de Design Thinking levam **semanas**.

Você tem 3 sprints e uma deadline na sexta.

Como você acelera isso sem perder qualidade?

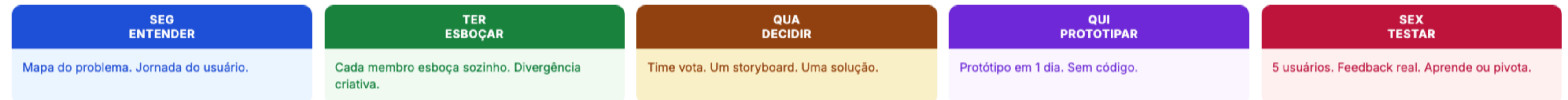
Google Ventures criou uma resposta para isso.

ATO 4 · DESIGN SPRINT

5 dias para responder uma pergunta crítica.

Sem reuniões infinitas. Sem código. Só protótipos. [Sprint – Jake](#)

[Knapp, Google Ventures \(2016\)](#)



DESIGN SPRINT · NUBANK EM 5 DIAS

Pergunta: como simplificar a abertura de conta?

SEG	TER	QUA	QUI	SEX
Jornada atual: 40 dias, 12 docs, 3 visitas. Mapa de dor.	Cada membro esboça: selfie + documento + formulário digital.	Storyboard: selfie → CPF → aprovação → cartão em casa.	Protótipo em Figma. 5 telas. 1 dia. Sem código.	5 usuários testam. 4 de 5 abrem conta sem ajuda. ✓

Na quinta-feira o Nubank tinha um protótipo. Na sexta, sabia se a ideia funcionava. Sem escrever uma linha de código.

PROTOTIPAGEM · TRÊS FIDELIDADES

Você testa uma ideia de R\$ 1 milhão com papel e caneta.

Alta Fidelidade

UI finalizada, clicável, dados reais

O que você entrega para o dev.

Média Fidelidade

wireframe digital — Balsamiq, Figma

Estrutura sem estética. Testa o fluxo.

Baixa Fidelidade

papel, post-it, rascunho à mão

Comunica a ideia. Custo: zero.

O CICLO COMPLETO

O protótipo não é o fim — é a entrada para a modelagem formal.



Protótipo Alta Fidelidade

Cada tela tem objetos com estado e comportamento.



Diagrama de Classes

Cada objeto vira uma classe. Cada relação vira uma associação.



Código

Agora sim. Implementação de algo já validado.

PROTOTIPAGEM · POR QUE VALE O INVESTIMENTO

Prototipagem não é perda de tempo. É a forma mais barata de descobrir que você está errado.

 Mudar um rascunho em papel

0 min

 Mudar um wireframe digital

~1 hora

 Mudar código em produção

dias ou semanas

Engenharia de Software é tudo que acontece **antes** do código.



**"O engenheiro ruim
começa a codar no dia 1."**

**"O engenheiro bom ainda
está desenhando no dia 5."**

SEMANA QUE VEM • 17/ABR

Você aprendeu a modelar a **estrutura do sistema.**

Próxima aula: o *comportamento*.

Sequência

Quem fala com quem, e quando

Atividades

Fluxos, decisões, paralelismo

Casos de Uso

Atores e funcionalidades

Traga o seu projeto em mente — você vai precisar deles.